# O&M runbooks — steady-state operations & maintenance

Operations and maintenance for a **deployed** Claude apps gateway. The `test-run-runbook` covers the initial deploy; this document covers what you do afterwards: rotating the certificate and secrets, refreshing the RDS CA bundle, pushing new Claude Code / image versions, responding to alarms, backup/restore, and teardown.

Every command uses the repo's own scripts and `deploy.env` variables — never hardcoded org values. Run operator commands from a host that has `deploy.env` filled in and AWS credentials for `us-gov-west-1` (the scripts source `scripts/common.sh`, which loads `deploy.env` and exports `AWS_REGION`). The three stack names come from `deploy.env`:

- `DB_STACK_NAME` — `01-database.yaml` (default `${NAME_PREFIX}-db`)
- `GATEWAY_STACK_NAME` — `02-gateway.yaml` (default `${NAME_PREFIX}`)
- `OBS_STACK_NAME` — `03-observability.yaml` (default `${NAME_PREFIX}-obs`)

## Verification status — read this first

Per the repo honesty rule (`.claude/rules/process.md`), each runbook is tagged by how far its steps have been exercised on real infrastructure. The source of truth is the fix log at the top of `security-review-2026-07.md` and the Status block in `CLAUDE.md`.

- **[VERIFIED-LIVE]** — exercised in the 2026-07 test run: certificate import into ACM, offline image builds on the hardened host, ALB + access logs, DB bootstrap + app-user auth, RDS TLS (verify-full via the OS trust store), and the endpoint-SG reachability model.
- **[NEEDS TEST-RUN CONFIRMATION]** — doc-verified against the scripts and templates but **not yet exercised end to end**: gateway steady state + login, secret rotation (DB app credential, Okta, Grafana, portal), Grafana Okta login, the activity archive, alarm firing, restore, teardown, and the entire **download portal** (stack `04`: live Okta round-trip with the groups claim, real-size streamed downloads, audit-log wiring). The asynchronous db-secret rotation event shape is the standing example (`CLAUDE.md`).

Treat a whole runbook's tag as the ceiling; individual steps call out anything more specific.

## Accepted risks (surfaced up front)

One deliberate, SSP-scoped decision affects operations and is stated here so it is not discovered mid-incident. A second, formerly-accepted risk (C2) was **withdrawn** by the live run and is noted so operators do not act on the old posture:

- **C2 — plaintext OTLP hop: WITHDRAWN, now a localhost sidecar.** The earlier posture (a plaintext-but-SG-scoped gateway→collector hop) was disproved live — the gateway rejects a non-HTTPS telemetry forward URL off localhost — so the collector was moved **into the gateway task as a sidecar** (2026-07-22). There is no network telemetry hop to secure; the collector shares the gateway's lifecycle. Operationally this means the collector is

**not** a separate service to restart or roll on its own — see runbook 6 (`security-review-2026-07.md` C2 + fix log). Two operational consequences of the default `TELEMETRY_FAIL_CLOSED=true` (AU-5): **a persistently failed or unhealthy collector stops the gateway task** (ECS replaces it; check the `otel/` log streams and the collector health check before suspecting the gateway itself), and every task stop drains gateway-first / collector-last with up to a 120 s flush window — expect task stops during rotations and deploys to take up to ~2 minutes longer than the pre-sidecar timings.
- **C9 — S3 Object Lock deferred.** The activity archive and ALB-log buckets rely on `DeletionPolicy: Retain` + bucket lifecycle, not Object Lock. A privileged operator can still delete archived objects; there is no WORM guarantee.

---

## 1. ALB TLS certificate rotation

*Trigger / Frequency:* The `${NAME_PREFIX}-certificate-expiry` CloudWatch alarm fires (`AWS/CertificateManager` `DaysToExpiry` ≤ `CERT_EXPIRY_ALARM_DAYS`, default 30) — or any unplanned re-issue (key compromise, CA change). **Imported ACM certificates do NOT auto-renew**, so this is a scheduled human task, roughly once per certificate lifetime (typically annually). Status: **[NEEDS TEST-RUN CONFIRMATION]** for the in-place replace + listener pickup; the initial import path is **[VERIFIED-LIVE]**.

*Preconditions:*

- Access to the enterprise CA to sign a new leaf (serverAuth EKU).
- `CERTIFICATE_ARN` and `GATEWAY_FQDN` set in `deploy.env` (the current cert's ARN — rotation replaces the certificate *in place* under the same ARN, so no stack update is needed and the ALB keeps its DNS name).
- Run the key/CSR steps on the PKI workstation; `import-enterprise-cert.sh` sources `common.sh` with `COMMON_SH_OPTIONAL_ENV=1`, so it works without a filled-in `deploy.env` (it only needs `set_env_var`, which no-ops with a warning if `deploy.env` is absent).

*Steps (exact commands):*

1. Generate a fresh key + CSR. The script does this under `umask 077` and removes any pre-existing key first, so the key is never briefly world-readable:

```
bash ./scripts/import-enterprise-cert.sh csr "$GATEWAY_FQDN" # writes ${GATEWAY_FQDN}.key.pem (0600) + ${GATEWAY_FQDN}.csr # SAN is exactly DNS:${GATEWAY_FQDN} (the corporate CNAME, not the ALB name) # Key type defaults to EC P-256; append rsa2048 (or rsa3072) if the CA # only issues RSA: ...csr "$GATEWAY_FQDN" rsa2048
```

1. Submit `${GATEWAY_FQDN}.csr` to the enterprise CA (serverAuth EKU). Collect the new leaf (`leaf.pem`) and the CA chain (`chain.pem`, intermediates first, root last).

2. **Publish the new SHA-256 fingerprint to developers BEFORE cutting over.** Rotation re-triggers Claude Code's first-connect trust prompt; developers who pinned the old fingerprint must be told the new one. The import step prints it, but you can print it ahead of the cutover from the leaf:

`bash openssl x509 -in leaf.pem -noout -fingerprint -sha256`

1. Replace the certificate **in place** under the existing ARN — the ALB listener picks up the new material with **no stack update**:

`bash ./scripts/import-enterprise-cert.sh import "$GATEWAY_FQDN" \ leaf.pem "${GATEWAY_FQDN}.key.pem" chain.pem \ --certificate-arn "$CERTIFICATE_ARN"`

The script re-validates SAN, serverAuth EKU, and key↔cert match before importing; on success it re-prints the fingerprint and the new expiry date, and persists `CERTIFICATE_ARN` back into `deploy.env` (unchanged on an in-place replace).

*Verification:*

- `scripts/verify-gateway.sh` — step 2/3 fetches the served cert, cross-checks its SHA-256 against the ACM-imported cert (when AWS creds are available), and prints the fingerprint to publish. A mismatch flags TLS interception (ZIA inspection), not a rotation success.
- Confirm the alarm clears: after ACM re-computes `DaysToExpiry` (daily, `Period: 86400`), `${NAME_PREFIX}-certificate-expiry` returns to `OK`.

*Rollback / recovery:* Re-import the previous leaf/key/chain under the same `--certificate-arn` (keep the outgoing material until the new cert is confirmed live). Because the ARN is stable, rollback is another in-place `import`; the ALB never changes DNS name or listener config.

*Notes & pitfalls:*

- **Do not** delete-and-recreate the ACM certificate or change the listener's `CertificateArn` in the template to rotate — that risks an ALB/listener update. In-place replace under the same ARN is the only sanctioned path.
- The SAN must be `DNS:${GATEWAY_FQDN}` (the corporate CNAME), never the `*.elb.amazonaws.com` name.
- **Test-account variant:** the self-signed shortcut (a leaf that is its own trust anchor, imported with no chain) lives in `test-run-runbook.md` §1 "Test-account shortcut — self-signed ALB cert". Production rotation always uses an enterprise-CA leaf + chain as above; the self-signed path is a cross-reference only.

---

## 2. Okta client-secret rotation (gateway, Grafana & portal)

*Trigger / Frequency:* Okta client secret expiry/rotation policy, suspected exposure, or an Okta app re-key. The gateway OIDC secret, the Grafana SSO secret, and (when stack `04` is deployed) the download portal's OIDC secret all ride the same `put_secret_and_roll` helper (hidden prompt → mode-600 `file://` write → forced ECS new-deployment). Status: **[NEEDS TEST-RUN CONFIRMATION]** (steady-state login not yet exercised).

*Preconditions:*

- **Coordinate with the Okta admin first.** Generate the new client secret in Okta and have its value in hand *before* running the script. Okta apps can hold two secrets during an overlap window — ask the admin to add the new secret without removing the old one, so there is no outage between "secret written to Secrets Manager" and "old secret retired in Okta".

- `GATEWAY_STACK_NAME` deployed (gateway secret); `OBS_STACK_NAME` deployed (Grafana secret); `PORTAL_STACK_NAME` deployed (portal secret).

*Steps (exact commands):*

- **Gateway OIDC secret** — reads `OktaClientSecretArn`, `ClusterName`, `ServiceName` from the gateway stack and rolls the gateway service:

```
bash ./scripts/set-okta-secret.sh # prompts "Okta client secret (input hidden):" — paste the NEW value
```

- **Grafana SSO secret** — reads `GrafanaOidcSecretArn` + `GrafanaServiceName` from the observability stack and `ClusterName` from the gateway stack (shared cluster), and rolls the Grafana service:

```
bash ./scripts/set-grafana-oidc-secret.sh # prompts "Okta client secret for Grafana (input hidden):" — paste the NEW value
```

If Grafana reuses the gateway's Okta app, this is the *same* secret value as `set-okta-secret.sh`.

- **Portal OIDC secret** (when stack `04` is deployed) — reads `PortalOidcSecretArn` + `PortalServiceName` from the portal stack and `ClusterName` from the gateway stack (shared cluster), and rolls the portal service:

```
bash ./scripts/set-portal-oidc-secret.sh # prompts hidden — paste the NEW value
```

If the portal reuses the gateway's Okta app (a documented option — `scripts/deploy.env.example`), this is the *same* secret value as `set-okta-secret.sh`; rotate **both** consumers in the same window.

*Verification:*

- Watch the roll to stable. The `put_secret_and_roll` helper echoes the exact `aws ecs wait services-stable …` command (with the resolved cluster/service) at the end of its run — copy and run that line.
- Gateway: `scripts/verify-gateway.sh` step 3/3 — the OAuth endpoints respond and issue a device `user_code`.
- Grafana: sign in at `https://${GATEWAY_FQDN}/grafana` via Okta.
- Portal: sign in at `https://${GATEWAY_FQDN}/portal` via Okta and reach the Team/Cost-Center page.

*Rollback / recovery:* If login breaks after the roll, re-run the same script and paste the **previous** secret value (Okta still honours it during the overlap window). Then investigate before retrying.

*Failure mode — rolling before Okta has the new value:* If you write a new secret to Secrets Manager and roll the service while Okta still expects the old one, the OAuth code exchange fails and logins break for everyone (Grafana with the login form disabled = total lockout). Always confirm the new secret is active in Okta first; keep the old secret valid until the new tasks are stable.

*Notes & pitfalls:*

- Never pass the secret on a command line — the scripts prompt for it (hidden) and write via a mode-600 `file://` temp file (`.claude/rules/security.md`).
- The template resources are placeholders (`REPLACE-ME-…`). Do **not** "rotate" by editing the template `SecretString` — that clobbers the live value on the next deploy (`.claude/rules/cloudformation.md`).

---

## 3. Database app-credential rotation

*Trigger / Frequency:* Normally **automatic** — the db-admin rotation Lambda (`${NAME_PREFIX}-db-rotation`) runs on the `APP_SECRET_ROTATION_DAYS` cadence (`deploy.env`, default 90; passed through as the `AppSecretRotationDays` template parameter). Manual triggers: suspected credential exposure, or recovery after a half-completed rotation. Status: **[NEEDS TEST-RUN CONFIRMATION]** — rotation is not yet exercised in steady state, and the asynchronous rotation event shape is doc-verified only (`CLAUDE.md`).

*Design (how it works — describe, don't guess):* The secret `${NAME_PREFIX}/db-app-user` alternates between two Postgres LOGIN users, `gateway_app` and `gateway_app_clone`, both of which assume the NOLOGIN owner role `gateway_owner` at login. Each rotation flips `AWSCURRENT` to the *other* user with a fresh password, so the **previous credential stays valid until the next rotation** — there is no window where a running task holds a dead credential. The four standard Secrets Manager steps (`docker/db-admin/app.py:rotate_handler`, semantics fixed by `tests/lambda/test_rotation.py`):

1. `createSecret` — put the pending value (other user + random password) at `AWSPENDING` (idempotent on retry).
2. `setSecret` — `ALTER ROLE <pending user> WITH PASSWORD …` as the RDS master.
3. `testSecret` — connect as the pending user and `SELECT 1`.
4. `finishSecret` — move `AWSCURRENT` to the new version (idempotent), then `forceNewDeployment` on the gateway service so new tasks fetch the new credential.

*Preconditions:* `DB_STACK_NAME` + `GATEWAY_STACK_NAME` deployed; the db-admin Lambda image is current.

*Steps (exact commands):*

- **Confirm the last rotation succeeded:**

```bash # Rotation metadata: last-rotated date, schedule, and which versions hold # which stages (expect one AWSCURRENT, and AWSPREVIOUS = the still-valid # prior credential). aws secretsmanager describe-secret --region "$AWS_REGION" \ --secret-id "$NAME_PREFIX/db-app-user" \ --query '{LastRotated:LastRotatedDate, RotationEnabled:RotationEnabled, Stages:VersionIdsToStages}'
```

# Rotation Lambda log group — look for "rotation finished; service roll requested" aws logs tail "/aws/lambda/$NAME_PREFIX-db-rotation" --region "$AWS_REGION" --since 100d ```

- **Trigger a manual rotation:**

`bash aws secretsmanager rotate-secret --region "$AWS_REGION" \ --secret-id "$NAME_PREFIX/db-app-user"`

(Rotation is asynchronous; the CLI returns immediately. Watch the log group above for the four steps.)

*Verification:*

- `describe-secret` shows a newer `LastRotatedDate` and `AWSCURRENT` on a new version id; `AWSPREVIOUS` points at the prior version.
- The `${NAME_PREFIX}-db-rotation-errors` alarm stays `OK` (threshold: ≥3 Lambda errors/hour — it tolerates the expected single Inactive-image error per scheduled rotation).
- Gateway tasks are stable after the `finishSecret` roll (`aws ecs wait services-stable … --services $NAME_PREFIX-gateway`).

*Rollback / recovery — half-completed rotation:* The design is retry-first: Secrets Manager retries a failed step, `finishSecret`'s label move is idempotent, and the prior credential remains valid, so a stuck rotation does **not** break running tasks — they keep working on the current credential.

- If a rotation is wedged, inspect `/aws/lambda/$NAME_PREFIX-db-rotation` for the failing step, fix the cause (common: the image Lambda is `Inactive` and the first invoke fails while Lambda re-optimizes — Secrets Manager's retries then complete it), and re-run `rotate-secret` to re-drive it.
- To abandon an in-flight `AWSPENDING` version without applying it, remove the `AWSPENDING` stage from that version (`update-secret-version-stage --remove-from-version-id …`); the live `AWSCURRENT` credential is untouched.
- **Do not** hand-edit `${NAME_PREFIX}/db-app-user` — it is Lambda-managed (least-privilege AC-6 design). Hand-editing desyncs the secret from the Postgres role passwords.

*Notes & pitfalls:*

- The gateway **never** uses the RDS master credential; the master secret is break-glass only (see runbook 7). Rotation `ALTER ROLE`s run as master inside the Lambda, not from any task.
- Flag per `.claude/rules/process.md`: the async rotation/EventBridge event shape is **doc-verified only until the test run confirms it**.

---

## 4. RDS CA bundle refresh

*Trigger / Frequency:* AWS rotates the RDS server CA (e.g. the `rds-ca-rsa2048-g1` family — the instance's `CACertificateIdentifier`), or you must move to a newer CA before an AWS-announced expiry. Rare (multi-year). Status: **[NEEDS TEST-RUN CONFIRMATION]** for the CA-change cutover; the image build + baked-bundle mechanism is **[VERIFIED-LIVE]**.

*Why this is an image rebuild, not a config flip:* both the gateway and the db-admin Lambda connect with `sslmode=verify-full`, and the driver trusts the **OS/ container trust store**, not `sslrootcert=` (proven in the test run — the driver ignores `sslrootcert=`). The RDS CA bundle is fetched at build time (`RDS_CA_BUNDLE_URL`, default the GovCloud truststore) and **baked into both images** (`docker/rds-ca-bundle.pem`, `docker/db-admin/rds-ca-bundle.pem`). A CA change therefore means: restage the bundle → rebuild **both** images with a **bumped immutable tag** → stack update that rolls the services and re-points the Lambda images.

*Preconditions:* Build host with Docker + egress to the RDS truststore (or a mirrored bundle), `KMS_KEY_ARN` set (CMK-encrypted ECR), and — if the CA identifier itself changes on the instance — a maintenance window (modifying `CACertificateIdentifier` on the DB may require a reboot).

*Steps (exact commands):*

1. **Rebuild the gateway image with a bumped tag** (tags are IMMUTABLE — a same-tag rebuild cannot be pushed). The build script re-fetches the RDS CA bundle every run:

`bash # bump the tag so the new bundle ships under a new immutable URI IMAGE_TAG="${CLAUDE_VERSION}-ca$(date +%Y%m%d)" ./scripts/build-and-push-image.sh # persists IMAGE_URI back into deploy.env`

1. **Rebuild the db-admin Lambda image with a bumped tag:**

`bash DBADMIN_VERSION="1.0.1" ./scripts/build-and-push-dbadmin.sh # persists DBADMIN_IMAGE back into deploy.env`

(Optionally override `RDS_CA_BUNDLE_URL` on both builds to pin a specific bundle for a controlled network.)

1. **Deploy the gateway stack** so the task definition and both Lambdas pick up the new image URIs and the service rolls (images **before** the stack update that expects them — `.claude/rules/scripts.md`):

`bash ./scripts/deploy-gateway.sh`

1. **Only if the instance CA identifier changes:** update `CACertificateIdentifier` on the RDS instance in `01-database.yaml` to the new CA and `./scripts/deploy-database.sh`. This is a property modification, not a replacement — verify it is not flagged as `Update:Replace` before applying (the stack policy denies replacement of `Database`).

*Verification:*

- Gateway + db-admin connect with `verify-full` against the new CA: `scripts/verify-gateway.sh` passes end to end, and a manual `aws secretsmanager rotate-secret --secret-id "$NAME_PREFIX/db-app-user"` completes its `testSecret` step (proves the rebuilt db-admin image trusts the new CA).
- `aws ecs wait services-stable … --services $NAME_PREFIX-gateway`.

*Rollback / recovery:* Redeploy with the previous `IMAGE_URI` / `DBADMIN_IMAGE` (both still exist under their old immutable tags in ECR) via `deploy.env` + `deploy-gateway.sh`. Because AWS RDS CA changes are additive (old + new CA trusted during the transition window), the old images keep validating until the old CA is retired.

*Notes & pitfalls:*

- Rebuild **both** images — a refreshed gateway with a stale db-admin image (or vice versa) leaves one side unable to validate `verify-full` after the CA fully cuts over.
- Do not attempt to fix a CA change by setting `sslrootcert=` — the driver ignores it. The only lever is the baked-in bundle.

---

## 5. Claude Code release update

*Trigger / Frequency:* A new pinned Claude Code release you want to distribute (security fix, feature, or a gateway-required minimum bump). Status: **[NEEDS TEST-RUN CONFIRMATION]** for the full mirror→build→distribute chain; offline image builds are **[VERIFIED-LIVE]**.

*Preconditions:*

- An egress host that can reach `downloads.claude.ai` (the mirror step). The laptops and the container build need **no** egress.
- `ANTHROPIC_GPG_KEY` set to Anthropic's release-signing public key so the manifest signature is verified. Verification **fails closed**: `ALLOW_UNVERIFIED_MANIFEST=1` is the only (deliberate, named) escape hatch and must not be the default (`.claude/rules/security.md`).

*Steps (exact commands):*

1. **Mirror the release** (linux-x64 for the image + win32-x64 for laptops):

```
bash ANTHROPIC_GPG_KEY=/path/to/anthropic-release-key.asc \ ./scripts/mirror/mirror-claude-release.sh 2.1.208 # verifies the GPG-signed manifest + per-binary SHA-256, writes # mirror/2.1.208/{claude,claude.exe,CHECKSUMS.txt}
```

1. **Rebuild the gateway image** (it embeds the linux binary). Set `CLAUDE_VERSION` to the new release in `deploy.env` first, then stage the verified binary and build:

```
bash cp mirror/2.1.208/claude docker/claude # deploy.env: export CLAUDE_VERSION="2.1.208" ./scripts/build-and-push-image.sh # tags the image 2.1.208, persists IMAGE_URI ./scripts/deploy-gateway.sh # rolls the gateway service onto it
```

1. **Publish to the download portal** (when stack `04` is deployed) — this is how developers self-serve the new version. Reuses the verified mirror output; uploads `claude.exe`, `manifest.json`, `CHECKSUMS.txt`, and the installer to the portal's CMK-encrypted artifacts bucket, then pins the portal to the new version:

```
bash ./scripts/publish-portal-release.sh 2.1.208 # deploy.env: export PORTAL_RELEASE_VERSION="2.1.208" (empty = CLAUDE_VERSION) ./scripts/deploy-download-portal.sh # only needed when the pinned version changes
```

1. **Distribute the Windows client (share/MDM route).** Stage `mirror/2.1.208/claude.exe` + `CHECKSUMS.txt` on the file share and install non-elevated per developer:

```
powershell .\client\Install-ClaudeCode.ps1 ` -BinaryPath \\fileserver\software\claude\2.1.208\claude.exe ` -Sha256 <win32-x64 checksum from CHECKSUMS.txt> ` -GatewayUrl https://<GATEWAY_FQDN> -DisableUpdates
```

1. **Forcing the upgrade (optional).** The installer is user-scope only and has no settings-push mode; a client-side version floor is a **managed setting**, delivered by GPO/MDM (see `client-config.md` §2). To raise the floor, bump `requiredMinimumVersion` in the managed-settings JSON the GPO already delivers (default floor is `2.1.195`, the gateway's minimum):

```
json {"forceLoginMethod":"gateway","forceLoginGatewayUrl":"https://<GATEWAY_FQDN>","forceRemoteSettingsRefresh":true,"requiredMinimumVersion":"2.1.208"}
```

Update the GPP Registry value `Settings` under `HKLM\SOFTWARE\Policies\ClaudeCode` (or the `%ProgramFiles%\ClaudeCode\managed-settings.json` file) with the new floor; clients pick it up on the next Group Policy refresh. The gateway also enforces its own minimum client version server-side regardless, so a below-floor client is refused at the gateway even without the GPO lock.

*Verification:*

- Mirror step: the script prints `checksum OK` per platform and `manifest signature OK`. A SHA-256 or signature mismatch aborts (non-zero exit) and removes the bad file.
- Gateway: `aws ecs wait services-stable … --services $NAME_PREFIX-gateway`, then `scripts/verify-gateway.sh`.
- Client: `claude --version` reports the new version; below-floor binaries refuse to start when `requiredMinimumVersion` is raised.
- Portal (if published): download a ZIP from `https://${GATEWAY_FQDN}/portal` and confirm it contains the new `claude.exe` and that the generated `install.cmd` carries the new version's SHA-256; the download appears in the portal audit log group (`/claude/${NAME_PREFIX}/portal-audit`).

*Rollback / recovery:* Redeploy the previous `IMAGE_URI` (old immutable tag still in ECR) via `deploy.env` + `deploy-gateway.sh`; lower `requiredMinimumVersion` back in the GPO managed-settings JSON if you raised the floor (`client-config.md` §2). Portal: set `PORTAL_RELEASE_VERSION` back to the prior version and re-run `deploy-download-portal.sh` (earlier `releases/<version>/` prefixes stay in the artifacts bucket). Keep the prior `mirror/<version>/` directory until the new release is confirmed across the fleet.

*Notes & pitfalls:*

- Update lockdown (`-DisableUpdates` → `DISABLE_UPDATES=1` + `DISABLE_AUTOUPDATER=1`) is what keeps users on the distributed version — do not drop it, or clients will self-update off the pinned build.

- **The installer is user-scope only** — it must run in the developer's own (non-elevated) context. A SYSTEM-context run is **refused outright** (it would install the binary into SYSTEM's profile and PATH, which no developer sees), and there is no settings-push mode: the installer writes only the user settings `env` block ( `DISABLE_UPDATES` / `DISABLE_AUTOUPDATER` , `OTEL_RESOURCE_ATTRIBUTES` , `NODE_EXTRA_CA_CERTS` ), never a machine/policy source. For a device-context binary push use the MDM "user" install behavior (Intune) or the download portal.
- **Gateway login REQUIRES the managed setting — an admin-channel concern, not the installer's.** `forceLoginMethod: "gateway"` + `forceLoginGatewayUrl` are what make the **Cloud gateway** login path appear at all; without them `/login` shows only the standard account picker with **no gateway option and no way for a user to type a gateway URL** (Anthropic's anti-phishing design). These keys are honored **only** from a **managed source** — `HKLM\SOFTWARE\Policies\ClaudeCode` or `%ProgramFiles%\ClaudeCode\managed-settings.json` — **never** from user `settings.json` or HKCU (a user-level `forceLoginMethod:"gateway"` is explicitly nulled by the binary), which is why the installer writes no policy source. `requiredMinimumVersion` rides the same managed JSON. On hardened fleets the `Policies` subtree is ACL-locked under STIG/CIS baselines, so the entry is delivered by GPO/MDM: a GPP Registry `REG_SZ` value `Settings` under `HKLM\SOFTWARE\Policies\ClaudeCode` , or a GPP Files copy of `managed-settings.json` to `%ProgramFiles%\ClaudeCode\` (Claude Code moved off `%ProgramData%` at v2.1.75; admin-write-only = tamper-resistant; **[BINARY-VERIFIED]** against the mirrored 2.1.211 binary). A developer **with local admin** can self-serve the entry once. Full AD-admin steps are in `client-config.md` §2; the AD/GPO request template is `ad-request-email.md`. The **binary install stays no-admin either way** — only the login config needs the managed setting.
- Never bypass GPG verification as a matter of routine; `ALLOW_UNVERIFIED_MANIFEST=1` is for a deliberately air-gapped one-off only.

---

## 6. Gateway / Grafana / collector image & stack updates

*Trigger / Frequency:* Any container change (Dockerfile fix, Grafana provisioning, a new ADOT collector release, task CPU/memory tuning, or a parameter change). Status: **[NEEDS TEST-RUN CONFIRMATION]** for steady-state rolls of Grafana and the collector sidecar; gateway image builds + deploys are **[VERIFIED-LIVE]**. Note the collector is a **sidecar in the gateway task** (not a standalone service since 2026-07-22): rolling it is a new **gateway task-definition revision**, done by re-running `deploy-gateway.sh`, not `deploy-observability.sh`.

*Model changes (* `OPUS_MODEL_ID` */* `SONNET_MODEL_ID` *) are a special case:* these parameters drive three things at once — the Bedrock routing, the scoped IAM / endpoint policies, and the **client model allowlist the gateway pushes via** `/managed/settings`. Changing them therefore changes what every developer's `/model` picker offers. Re-run `deploy-gateway.sh`; connected clients pick the new allowlist up on their next settings fetch (re-login if it lags). Verify with `/model` on a real client before calling it done.

*Preconditions:* Build host with Docker; `KMS_KEY_ARN` set; the relevant stack already deployed. **Rebuild and push the image BEFORE the stack update that expects it** ( `.claude/rules/scripts.md` ).

*Steps (exact commands):* bump the immutable tag → build/push → deploy.

- **Gateway:** `IMAGE_TAG=<new> ./scripts/build-and-push-image.sh` → `./scripts/deploy-gateway.sh`.
- **Grafana:** `GRAFANA_IMAGE_TAG=<new> ./scripts/build-and-push-grafana.sh` → `./scripts/deploy-observability.sh`. Notes for version bumps (learned on the 11.5.1 → 13.1.1 upgrade, 2026-07-25): the OSS image is `grafana/grafana` (the `grafana/grafana-oss` Docker Hub repo froze at 12.4);

the build script also stages the `grafana-amazonprometheus-datasource` plugin into the image (SigV4 left the core prometheus datasource in 13.1 and the task has no egress to install plugins at boot) — bump `AMP_PLUGIN_VERSION` + `AMP_PLUGIN_SHA256` together in the script when updating it; and expect a **one-time re-login for all Grafana users** on the first post-upgrade start (external OAuth sessions are re-linked — `improvedExternalSessionHandling`, default-on since 12.1).

- **ADOT collector (sidecar):** `ADOT_VERSION=<vX.Y.Z> ./scripts/mirror/mirror-collector.sh` (mirrors + pins `COLLECTOR_IMAGE` by digest) → `./scripts/deploy-gateway.sh` (the sidecar lives in the gateway task, so the collector rolls with a new gateway task-def revision — **not** an observability-stack update).
- **db-admin Lambda:** `DBADMIN_VERSION=<new> ./scripts/build-and-push-dbadmin.sh` → `./scripts/deploy-gateway.sh`.
- **Download portal:** `PORTAL_VERSION=<new> ./scripts/build-and-push-portal.sh` → `./scripts/deploy-download-portal.sh`.

Each build script persists its new URI/tag into `deploy.env`, so the matching `deploy-*.sh` picks it up with no copy-paste.

*Update-safety invariants (must hold on every stack update — `.claude/rules/cloudformation.md`):*

- **The ALB and RDS instance must never be replaced by a routine update.** Both are protected three ways — deletion protection, fixed physical names, and a **stack policy** (set by `deploy-gateway.sh` / `deploy-database.sh`) denying `Update:Replace`/`Update:Delete` on `LoadBalancer` / `Database`. Do not remove any layer. `deploy-gateway.sh` deploys with `--disable-rollback` by default (`CFN_DISABLE_ROLLBACK=true`) precisely so a failed create keeps the protected ALB rather than attempting an impossible rollback delete.
- **Cross-stack exports are locked while imported.** 01 exports the CMK, DB endpoint, master-secret ARN, and client SG to 02; 02 exports SGs, the listener, and the cluster to 03. You cannot change an exported value in place while a downstream stack imports it — encryption-at-rest and resource names are day-one decisions.
- **Placeholder `SecretString` resources must not be touched.** `OktaClientSecret`, `GrafanaOidcClientSecret`, `PortalOidcClientSecret`, and `DbAppUserSecret` hold placeholder/managed values; editing the `SecretString` literal (or the resource Name/Description) re-applies the placeholder and clobbers the live secret. Rotate via the scripts (runbooks 2–3), never the template.
- `TaskCpu`/`TaskMemory` **must stay a valid Fargate pairing** — the template's `Rules` section asserts this; an invalid combo fails deploy with an opaque error. Change them via `TASK_CPU`/`TASK_MEMORY` in `deploy.env`.

*Verification:* `aws ecs wait services-stable` for the affected service (`$NAME_PREFIX-gateway`, `$NAME_PREFIX-grafana`, or `$NAME_PREFIX-portal` — there is no separate collector service; a collector roll is a `-gateway` roll); `scripts/verify-gateway.sh` for the gateway; Grafana login + dashboards for Grafana. For a collector-sidecar change, confirm the gateway task runs both containers (`gateway` + `otel-collector`) and metrics still reach AMP. Confirm the CloudFormation events show `UPDATE_COMPLETE` with **no** replacement of `LoadBalancer` or `Database`.

*Rollback / recovery:* Redeploy the prior image URI/tag from `deploy.env` (old immutable tags persist in ECR). For a parameter regression, re-run the deploy script with the previous `deploy.env` values. If a gateway create/update lands in `*_FAILED` with `--disable-rollback`, fix the cause and re-run `deploy-gateway.sh` — the deploy **continues** from where it failed rather than tearing down the protected ALB.

*Notes & pitfalls:* A same-tag rebuild cannot be pushed (immutable repos) and an unchanged image URI leaves the service/Lambda on old code — always bump the tag.

---

## 7. Secrets inventory & break-glass

*Trigger / Frequency:* Reference during audits, incident response, or before any secret change. Status: **[NEEDS TEST-RUN CONFIRMATION]** for the break-glass master path.

*Secrets inventory (all CMK-encrypted with the CMK from 01):*

| Secret (Name) | Stack | Rotation | How to rotate |
|---|---|---|---|
| RDS master `${NAME_PREFIX}` DB (`Database.MasterUserSecret`) | 01 | **Automatic**, RDS-managed, every 7 days | RDS-managed; break-glass use → force-rotate (below) |
| `${NAME_PREFIX}/db-app-user` | 02 | **Automatic**, alternating-user Lambda, `AppSecretRotationDays` (default 90) | Runbook 3 |
| `${NAME_PREFIX}/oidc-client-secret` (Okta) | 02 | **Manual** | `scripts/set-okta-secret.sh` (runbook 2) |
| `${NAME_PREFIX}/jwt-secret` (session signing) | 02 | **Not rotated automatically** (`GenerateSecretString` at create) | Manual — see below |
| `${NAME_PREFIX}/grafana-oidc-client-secret` | 03 | **Manual** | `scripts/set-grafana-oidc-secret.sh` (runbook 2) |
| `${NAME_PREFIX}/grafana-admin-password` | 03 | **Not rotated** (break-glass; login form disabled) | Regenerate manually (below) |
| `${NAME_PREFIX}/portal-oidc-client-secret` | 04 | **Manual** | `scripts/set-portal-oidc-secret.sh` (runbook 2) |
| `${NAME_PREFIX}/portal-session-secret` (cookie signing) | 04 | **Not rotated automatically** (`GenerateSecretString` at create) | Same file-based pattern as the JWT secret (below), then roll `$NAME_PREFIX-portal`; rotation invalidates portal sessions (users just re-login) |

*Gateway JWT secret (manual rotation).* The template describes rotation as "prepend new value, roll, remove old." Whether the gateway honours two overlapping signing keys is **doc-verified only — needs test-run confirmation**; until confirmed, treat a JWT rotation as session-invalidating (active sessions below `SessionTtlHours`, default 1h, are dropped and users re-login). Rotate by writing a new value via the same safe pattern the helper uses, then forcing a roll:

```
JWT_ARN=$(aws cloudformation describe-stack-resources --region "$AWS_REGION" \
  --stack-name "$GATEWAY_STACK_NAME" --logical-resource-id JwtSecret \
  --query 'StackResources[0].PhysicalResourceId' --output text)
```

```
# generate + write without ever putting the value on argv:
f=$(mktemp); chmod 600 "$f"; aws secretsmanager get-random-password \
  --region "$AWS_REGION" --password-length 48 --exclude-punctuation \
  --query RandomPassword --output text > "$f"
aws secretsmanager put-secret-value --region "$AWS_REGION" \
  --secret-id "$JWT_ARN" --secret-string "file://$f"; rm -f "$f"
aws ecs update-service --region "$AWS_REGION" \
  --cluster "$(aws cloudformation describe-stacks --region "$AWS_REGION" \
    --stack-name "$GATEWAY_STACK_NAME" \
    --query "Stacks[0].Outputs[?OutputKey=='ClusterName'].OutputValue" --output text)" \
  --service "$NAME_PREFIX-gateway" --force-new-deployment
```

*Grafana admin password (break-glass regenerate).* Same file-based pattern against `${NAME_PREFIX}/grafana-admin-password`, then roll `$NAME_PREFIX-grafana`. The login form stays disabled unless `GRAFANA_DISABLE_LOGIN_FORM=false`; day-to-day access is Okta SSO.

*Break-glass — RDS master secret.* The master credential is **break-glass ONLY**; no task ever injects it (the gateway uses `${NAME_PREFIX}/db-app-user`). Use it only for direct DBA access during an incident.

*Steps:*

1. Read it without echoing to the terminal history:

`bash MASTER_ARN=$(aws cloudformation describe-stacks --region "$AWS_REGION" \ --stack-name "$DB_STACK_NAME" \ --query "Stacks[0].Outputs[?OutputKey=='DBMasterSecretArn'].OutputValue" --output text) # inspect keys interactively; avoid persisting the value anywhere aws secretsmanager get-secret-value --region "$AWS_REGION" \ --secret-id "$MASTER_ARN" --query SecretString --output text`

1. Perform the minimum necessary DBA action, from a host that can reach the DB (in-VPC operator host; the DB is `PubliclyAccessible: false`).

2. **Immediately afterwards — rotate the master back** so the just-exposed value is retired, and **record the use** (who/when/why) for the audit trail:

`bash aws secretsmanager rotate-secret --region "$AWS_REGION" --secret-id "$MASTER_ARN"`

*Verification:* `describe-secret` on the master shows a fresh `LastRotatedDate` after the forced rotation; the gateway is unaffected (it never used the master).

*Notes & pitfalls:* Treat the activity-log stream as highly sensitive (bash commands, tool inputs, file paths per user) — opt-in, IAM-only, CMK-encrypted, SIEM-flagged; never widen its access surface. Never put any secret value on a command line (`--secret-string <value>` leaks via `ps`/`/proc`); always the mode-600 `file://` pattern above.

---

## 8. Backup & restore

*Trigger / Frequency:* Reference for DR planning; act on data-loss/corruption or before a risky change. Status: **[NEEDS TEST-RUN CONFIRMATION]** for the restore path.

*Posture:*

- **RDS automated backups** — `BackupRetentionPeriod = BackupRetentionDays` (`01-database.yaml`, default 14, max 35). Daily automated snapshots + PITR within the window. `DeletionPolicy: Snapshot` / `UpdateReplacePolicy: Snapshot` → a stack delete/replace takes a **final snapshot** rather than destroying data. `DeletionProtection: true` blocks accidental instance deletion.
- **ALB access logs** — `AlbLogsBucket` (SSE-S3; ELB delivery does not support KMS — this is the one documented CMK exception), `DeletionPolicy: Retain`, lifecycle expiry at `AlbLogRetentionDays` (default 90).
- **Activity archive** — `ActivityArchiveBucket` (CMK-encrypted, `DeletionPolicy: Retain`), lifecycle expiry at `ActivityArchiveRetentionDays` (default 731 ≈ 2y). The CloudWatch window group `/claude/${NAME_PREFIX}/activity` retains `ActivityLogWindowDays` (default 14) before the Firehose→S3 chain is the durable copy. Idle (no cost) until the gateway sets `ForwardActivityLogs=true`.
- **AMP** — the workspace is `DeletionPolicy: Retain` so a routine 03 recreate does not destroy metrics history.

*Take an on-demand snapshot (before risky changes):*

```
aws rds create-db-snapshot --region "$AWS_REGION" \
  --db-instance-identifier "$NAME_PREFIX-store" \
  --db-snapshot-identifier "$NAME_PREFIX-store-preop-$(date +%Y%m%d%H%M)"
```

*Restore — understand the blast radius first.* **A replaced RDS instance is an EMPTY database, not a restore** (`.claude/rules/cloudformation.md`), and the DB endpoint is a cross-stack export imported by 02, which is **locked while imported**. You cannot restore in place by pointing the stack at a snapshot without disturbing that export. Restore is therefore effectively a **teardown + restore**, not an update:

1. Restore the snapshot to a **new** instance out-of-band to validate the data (`aws rds restore-db-instance-from-db-snapshot …`), or use PITR.
2. To make the restored data the live store, the sanctioned path is to bring the database stack back from the snapshot (RDS snapshot-based restore) with the same `DBInstanceIdentifier`/exports so 02 re-imports the endpoint — which, given the export lock and deletion protection, means an orchestrated teardown of 02 first, restore of 01 from the snapshot, then redeploy of 02/03. Plan this as a maintenance-window operation, not a routine update.

*Verification:* `aws rds describe-db-snapshots` shows the expected automated + manual snapshots; a test restore to a scratch instance connects and shows the expected schema/rows. After a real restore, `scripts/verify-gateway.sh` passes and the gateway serves logins.

*Rollback / recovery:* Snapshots are immutable point-in-time copies — a failed restore attempt is retried against another snapshot; the source snapshots are unaffected. Keep the pre-op on-demand snapshot until the operation is confirmed.

*Notes & pitfalls:* Per C9 (accepted risk), neither S3 bucket uses Object Lock — archived logs are deletable by a privileged operator. If tamper-evidence becomes a requirement, revisit C9 in the security review.

---

## 9. Alarm response

*Trigger / Frequency:* On alarm (routed to `ALARM_SNS_TOPIC_ARN` when set — otherwise the alarms exist but have no action). Status: **[NEEDS TEST-RUN CONFIRMATION]** — alarms are defined but have not fired in the test run.

*Alarms defined in the templates:*

1. `${NAME_PREFIX}-certificate-expiry` (`02-gateway.yaml`) — `AWS/CertificateManager` `DaysToExpiry` ≤ `CERT_EXPIRY_ALARM_DAYS` (default 30). *Response:* the imported cert is approaching expiry and will **not** auto-renew → execute **runbook 1** (re-issue from the enterprise CA, publish the new fingerprint, in-place `import --certificate-arn`). Alarm clears once ACM recomputes `DaysToExpiry` (daily).

2. `${NAME_PREFIX}-db-rotation-errors` (`02-gateway.yaml`) — `AWS/Lambda` `Errors` ≥ 3 in one hour on `${NAME_PREFIX}-db-rotation`. *Response:* rotation is erroring repeatedly (running tasks are still fine on the current credential, but the rotation SLA is at risk). Inspect `/aws/lambda/$NAME_PREFIX-db-rotation`, identify the failing step, and follow **runbook 3** recovery. The threshold intentionally tolerates the single expected Inactive-image error per scheduled rotation.

3. `${NAME_PREFIX}-missing-telemetry` (`03-observability.yaml`) — no samples ingested into the AMP workspace for `MISSING_TELEMETRY_ALARM_MINUTES` (default 15) consecutive minutes (`AWS/Usage` `ResourceCount` / `Resource=IngestionRate` scoped to the workspace; missing data = breaching, because AMP stops emitting the metric when nothing arrives). This is the **end-to-end backstop for the fail-closed telemetry posture** — container health only proves the collector is alive; this proves data is landing. *Response — triage in this order:* (1) is the gateway service running at all? (a full outage also silences telemetry — check the service first, this alarm may be a symptom); (2) `otel/`-prefixed log streams in the gateway log group for collector/export errors; (3) collector container health in the task detail (fail-closed replaces the task; fail-open leaves it running UNHEALTHY); (4) task-role `aps:RemoteWrite` + `aps-workspaces` endpoint reachability; (5) AMP-side rejection — check `AWS/Prometheus` `DiscardedSamples` for the workspace (throttling/validation). *Expected (not actionable) firings:* between the 03 deploy and the telemetry-enabled 02 re-run, and during deliberate gateway downtime.

4. `${NAME_PREFIX}-missing-activity-logs` (`03-observability.yaml`, **off by default**) — only created when `ACTIVITY_LOGS_ALARM_MINUTES` > 0. No events delivered to the activity audit log group for that many minutes (`AWS/Logs` `IncomingLogEvents`, missing = breaching). This is the audit-stream (AU-2/AU-12) counterpart to alarm 3 — alarm 3 watches *metrics*, this watches the *audit logs* pipeline, which alarm 3 and the collector health check cannot see. *Response:* same triage as alarm 3, plus check the collector's `awscloudwatchlogs` exporter for errors in the `otel/` streams. **Before declaring a fault, confirm the fleet was actually active** — this stream is intermittent, so genuine idleness also reads as silence; correlate with the usage metrics (a live metrics stream

5. dead audit stream = real fault; both quiet = probably just idle). Left off by default for exactly this reason; enable only on continuously-active fleets with a window longer than the longest expected quiet gap.

*No other CloudWatch alarms are defined in the templates.* Operational surfaces to watch manually: ECS service events / `services-stable`, and the gateway (which now also carries the collector sidecar's `otel`-prefixed streams) and Grafana log groups.

*General verification after responding:* confirm the alarm returns to `OK` (`aws cloudwatch describe-alarms --alarm-names <name> --query 'MetricAlarms[0].StateValue'`).

*Known landing-zone gotcha — ALB access-log AccessDenied.* If ALB access-log enablement fails `AccessDenied` on a bucket policy that is correct, **suspect a landing-zone auto-remediation rewriting the ALB's log config before suspecting the bucket policy** (test-run lesson). The bucket policy already grants both ELB delivery principals; the transient post-deploy log-enable variant was removed once the environment auto-remediation was exempted. Get the auto-remediation exempted for this ALB rather than re-editing the policy.

*Rollback / recovery:* Alarm response is corrective, not stateful — there is nothing to roll back beyond the underlying runbook's own recovery.

---

### Is the gateway capturing usage at all? (dump Postgres)

*Run:* `scripts/diagnostics/dump-usage.sh` (in-VPC host or bastion; needs IAM to read `<prefix>/db-app-user`). Read-only, connects the same way the gateway does (app-user secret + RDS CA, verify-full).

*Dependencies:* `botocore` (already present - the AWS CLI ships it) and `pg8000`. On an offline/hardened box, install pg8000 from the repo's vendored wheels rather than PyPI:

```
pip install --no-index --find-links docker/db-admin/vendor pg8000
```

No `boto3` needed.

*Two security controls gate this - both by design, neither needs a template change:*

1. **Network (RDS security group).** The DB admits only members of the `<prefix>-db-client-sg` SG (stack 01 output `DBClientSecurityGroupId`), described as "attach to workloads that may reach the gateway database" - that is exactly this use. **Attach that SG to your in-VPC admin instance's ENI** (additive; SG rules union, so it disturbs nothing else). The box must be an EC2 instance IN the VPC - security groups don't apply from outside it, so an off-VPC host needs a bastion. Do NOT widen the DB SG ingress; use the membership SG.

2. **Secret decrypt (IAM + KMS).** The app-user secret is CMK-encrypted, so the operator role needs BOTH `secretsmanager:GetSecretValue` on `<prefix>/db-app-user` AND `kms:Decrypt` on the CMK (scoped `kms:ViaService=secretsmanager.<region>.amazonaws.com`). The `kms:Decrypt` is the non-obvious half - the same CMK trap that produced 403s for Grafana/AMP.

The tool uses the **app-user** secret (least-privilege, gateway DB only), never the RDS master - the master stays break-glass. The app user auto-assumes `gateway_owner` at connect (`ALTER ROLE ... IN DATABASE ... SET role`), so it can read these tables with no extra grant - the same path the gateway uses.

*What Postgres holds - and does not.* The gateway does **not** store per-request token counts in Postgres. It stores: - `spend` - aggregate **cents per principal per period** (derived from tokens via the model rate table). This is the token-perspective capture, summed. - `principal_emails` - identity and the **Okta groups** the gateway resolved. - `spend_limits` / `admin_audit` - caps and the admin mutation log. Raw per-request token metrics live only in AMP - use `diagnose-telemetry.sh`.

*Reading it:* - `spend` empty while inference is happening -> the gateway is metering nothing. Check the gateway logs for `spend meter has no exact rates for model` (the served model IDs are not in the rate table). - `principal_emails` present but every `groups` is NULL/empty -> the gateway is not getting the Okta groups claim. This is the usual reason the Grafana `user_groups` filter shows nothing AND per-group spend caps match nobody. Fix at the Okta side (see `okta-request-email.md`), not in the gateway.

---

## Client usage metrics not reaching AMP / empty Grafana dashboard

*Symptom:* activity logs arrive, but `claude_code_*` panels in Grafana are empty.

*Run:* `scripts/diagnostics/diagnose-telemetry.sh`. It reads the ALB access logs (client -> `/managed/settings` and client -> `/v1/metrics`) and then queries AMP directly over SigV4, so it distinguishes four different failures that all look identical in Grafana:

| Evidence | Meaning |
|---|---|
| no `/managed/settings` | enrollment - clients never got the OTLP env vars |
| settings but no `/v1/metrics` | push lands, export never starts |
| exports, AMP empty of `otelcol_*` too | remote_write not landing at all |
| exports, `otelcol_*` present, no `claude_code_*` | check `otelcol_exporter_prometheusremotewrite_failed_translations` FIRST (see below) |
| `claude_code_*` present | **ingestion is fine** - it is a dashboard/query problem |

That last row is the easy one to misread: the dashboard filters on `team` / `cost_center` / `user_groups`, so if those labels are absent the panels render empty even though the data is there. The script prints the labels actually present on `claude_code_cost_usage` for exactly this reason. `team`/`cost_center` come from the installer's `OTEL_RESOURCE_ATTRIBUTES`; `user_groups` is stamped by the gateway from the Okta claim.

*Note:* read a 403 from the AMP query by its body. **SignatureDoesNotMatch** ("the request signature we calculated does not match") is a client-side SIGNING/ encoding bug, not permissions - do not chase key policies. (Historic instance: `quote_plus` encoded PromQL spaces as `+`, an ambiguous byte under SigV4, so only the space-carrying throughput queries failed; fixed 2026-07-24, regression-tested.) A 403 **without** that phrasing is the operator-role gap (missing `aps:QueryMetrics`, or the CMK `kms:Decrypt` trap from 2026-07-23) - and either way it says nothing about whether ingestion is working.

> **The silent-drop trap (root-caused 2026-07-24, live).** Claude Code clients export **delta**-temporality sums by default, and the sidecar's `prometheusremotewrite` exporter **cannot represent delta** - it drops those points at *translation*. The dropped points are **still counted in** `otelcol_exporter_sent_metric_points`, `send_failed` stays 0, and nothing is logged (reproduced on the pinned ADOT v0.43.0) - so every throughput counter looks healthy while zero client metrics reach AMP. The giveaway is `otelcol_exporter_prometheusremotewrite_failed_translations` climbing in step with client activity. Fix (02, 2026-07-24): the gateway pushes `OTEL_EXPORTER_OTLP_METRICS_TEMPORALITY_PREFERENCE=cumulative` to every client via `/managed/settings`; clients pick it up on their next settings fetch. A `deltatocumulative` processor in the sidecar was deliberately rejected: with `DesiredCount: 2` the ALB round-robins one client's exports across both sidecars, whose independent delta->cumulative reconstructions of the same series would conflict. (Verified on both v0.43.0 and the deployed v0.49.0 pin. The v0.49 deprecation warning about `add_metric_suffixes` is harmless - the key is still honored. If ever migrating, the equivalent is `translation_strategy: UnderscoreEscapingWithoutSuffixes` - the WithOUT variant; the WithSuffixes variant would re-add unit/type suffixes and break the dashboard names.)
>
> **Dashboard queries reworked for per-session series (2026-07-24).** Keeping `session.id` split cost/usage into one series per session, which broke the dashboard's `increase(...[window])` panels: `increase()` needs >=2 samples per series and returns NOTHING for a short/sparse session, so panels read ZERO while work is happening. The panels were reworked (validated against the pinned ADOT v0.49.0 -> Prometheus, not live AMP): - cumulative TOTALS (cost/token stat tiles, top-users table): `sum(max_over_time(metric[$__range]))` - each session's counter peak is its total; exact, never zero on sparse sessions. - cumulative RATES (time-series "by team/model/..."): `sum by (X) (max_over_time(m[1h]) - min_over_time(m[1h]))` - each session's rise within the trailing hour; a single-sample session contributes 0 instead of emptying the whole panel. - COUNTS (Sessions, Active users): `count(group by (session_id|user_email) (count_over_time({__name__=~"claude_code_.+", ...}[window])))` - distinct ids seen in the window; robust to single-sample and already-ended (stale) sessions, unlike the old counter-`increase()` and instant counts. The dashboard JSON is baked into the Grafana image, so **rebuild + push the Grafana image (bump GRAFANA_IMAGE_TAG) and re-run 03** to pick up the change. Caveat: sessions spanning the window's start edge slightly over-attribute (their pre-window spend counts in the window). For exact accounting the gateway's Postgres `spend` table remains authoritative.
>
> **Session labels (2026-07-24).** `session.id` is KEPT as a metric label. It was previously deleted for cardinality, but each session's counters start at 0, so concurrent sessions from the same user+model interleaved onto one series as a sawtooth - `increase()` read every alternation as a counter reset and inflated spend drastically (observed live on the dashboard). With the label kept, each session is its own monotonic series and the sum-by-team/user panels are exact with no query changes. Cardinality is bounded by CONCURRENT sessions (stale series age out of AMP's active-series count within minutes). If a fleet ever grows to where that matters, resize deliberately - do not silently re-add the delete, or the sawtooth inflation returns.

---

### Developers bounced back to browser login every ~1 hour

*Symptom:* a developer is forced through the full browser SSO at a fixed interval matching `SESSION_TTL_HOURS` (default 1h). Often paired with: after the bounce, `/login` shows the **default account picker** instead of the gateway, and only closing and reopening Claude Code restores the forced-gateway login.

*Cause:* the gateway refreshes its session JWT silently using an UPSTREAM Okta **refresh token** ( `[gateway-refresh] refreshed gateway JWT` in the gateway logs on success). If Okta issued no refresh token, there is nothing to refresh, so the session simply dies at the TTL. Okta issues a refresh token only when BOTH are true on the app: the **Refresh Token grant type** is enabled, and `offline_access` is a granted scope. Our clients request `offline_access`, but if the Okta app was set up for Authorization Code only, Okta drops it silently. The `/login`-shows-the-picker follow-on is a consequence: `forceLoginMethod: "gateway"` is applied at process startup, and the mid-session re-login path does not re-assert it — restarting re-reads the managed setting. Fixing the refresh token removes the mid-session re-login entirely, so the picker problem disappears with it.

*Fix (Okta admin, no redeploy):* on the gateway app, enable the **Refresh Token** grant type alongside Authorization Code; confirm `offline_access` is granted; on an org authorization server, confirm its refresh-token policy issues them. Then log in fresh once. See `docs/requests/okta-request-email.md` (updated to request both grants). This is TTL-independent — lowering `SESSION_TTL_HOURS` only changes how fast revocation propagates, not whether refresh works.

*Confirming:* success shows `[gateway-refresh] refreshed gateway JWT` in the gateway logs; failure shows `[gateway-refresh] IdP rejected refresh token; clearing it` or `OAuth session expired and could not be refreshed`.

---

### Telemetry forward failing with `ECONNREFUSED_SSRF`

*Symptom:* gateway logs `forward to http://localhost:4318 failed: Error: ECONNREFUSED_SSRF: blocked (cloud metadata / link-local): localhost -> 127.0.0.1`. Metrics stop reaching AMP and the `missing-telemetry` alarm fires, while the gateway itself keeps serving traffic.

*Cause:* the gateway blocks loopback addresses by default in its SSRF guard. The sidecar is reached over loopback, so the forward is rejected. Note the gateway *requires* a loopback host for a non-HTTPS `forward_to` — so the two rules conflict unless the override below is set.

*Fix:* `CLAUDE_GATEWAY_ALLOW_LOOPBACK=1` on the gateway container. The template sets this whenever telemetry is enabled; if you see this error, the running task predates that change — re-run `deploy-gateway.sh` and confirm the new task definition carries the variable:

```
aws ecs describe-task-definition --task-definition <gateway-td> \
  --query "taskDefinition.containerDefinitions[?name=='gateway'].environment[?name=='CLAUDE_GATEWAY_ALLOW_LOOPBACK']"
```

*Security note:* the override re-permits only `loopback` and `unspecified`. Link-local (169.254.0.0/16, including EC2 IMDS `169.254.169.254`), `100.100.100.200`, and `fd00:ec2::254` remain blocked — probe-verified. It does suppress the startup "pod can reach cloud metadata endpoint" warning, which is a diagnostic only; the egress controls are unchanged.

---

## 10. Spend caps (per-user / per-group cost limits)

*Moved:* spend management now has its own runbook — `cost-controls.md`. It covers the enforcement model (the `admin:` master switch; caps as `spend_limits` rows), setting caps via the portal admin page or the break-glass `scripts/set-spend-limit.sh` CLI, monitoring spend (Grafana dashboard, AMP queries, Postgres ground truth), what a capped developer sees, the **fail-closed spend-store outage** incident runbook (an RDS problem halts all inference fleet-wide — deliberate 2026-07-24 decision), the audit trail, and known gaps. This section number is kept so existing references to "om-runbooks §10" keep resolving.

---

## 11. Client recovery — Claude Code won't start after `/logout`

*Trigger / Frequency:* A developer reports that `claude` exits at launch with **"Unable to connect to Anthropic services"**, and `claude auth login` refuses with *"forceLoginMethod is 'gateway' in managed settings; run interactive /login to authenticate."* Typically right after they ran `/logout`. Status: **[BINARY-VERIFIED against `mirror/2.1.211/claude`, 2026-07-24; NEEDS TEST-RUN CONFIRMATION of the recovery steps on a real laptop]** — the mechanism was read out of the shipped build during the live run; the fix has not yet been re-exercised end to end.

*Why it happens (this deployment makes a benign check fatal):* the two logout paths are **not** equivalent.

- `/logout` **(slash command)** clears credentials **and** onboarding (`clearOnboarding: true`): it sets `hasCompletedOnboarding = false` in `%USERPROFILE%\.claude.json` and **deletes the whole credential store** — including `enterpriseGateway` (the session) *and* `gatewayTrust` (the pinned TLS fingerprint).
- `claude auth logout` **(CLI subcommand)** clears credentials only (`clearOnboarding: false`) and leaves the client able to start.

After `/logout`, the next launch re-enters the **onboarding** flow. Claude Code derives the login method from *whether gateway credentials exist* — with them deleted it resolves to `firstParty`, which flips OAuth onboarding on and puts a **connectivity preflight first in the step list**. That preflight is absent while a gateway session exists, which is why the failure only appears after a logout. It `GET`s `https://api.anthropic.com/api/hello` **and** `https://platform.claude.com/v1/oauth/hello`, requires **HTTP 200 on both**, and on failure prints the error and exits 1 — **before** the gateway login screen is ever drawn. Those two hosts are fixed in the build: the gateway URL never substitutes for them. On a gateway-only egress path they are unreachable by design, so the developer is deadlocked — the CLI dies before login, and `claude auth login` is barred by the managed policy.

*Confirming it is this and not a gateway outage* (a non-200 — including a Zscaler block page — is the failure):

```
curl.exe -sS -o NUL -w "%{http_code}`n" https://api.anthropic.com/api/hello
curl.exe -sS -o NUL -w "%{http_code}`n" https://platform.claude.com/v1/oauth/hello
```

*Steps (run as the affected developer; no admin rights needed):* restore the onboarding flag, which removes the preflight step from the startup path.

```
$p = "$env:USERPROFILE\.claude.json"
Copy-Item $p "$p.bak"          # this file also holds project history — back it up
```

```
$j = Get-Content $p -Raw | ConvertFrom-Json
if ($j.PSObject.Properties.Name -contains 'hasCompletedOnboarding') {
  $j.hasCompletedOnboarding = $true
} else {
  $j | Add-Member -NotePropertyName hasCompletedOnboarding -NotePropertyValue $true
}
$j | ConvertTo-Json -Depth 100 | Set-Content $p -Encoding utf8
```

Then open a **new** terminal, run `claude`, and run `/login` — the locked "Cloud gateway" screen appears with the URL pre-filled (§1.2 of `client-config.md`).

*Verification:* `claude` starts to a prompt instead of exiting; `/login` completes the Okta round-trip; `/model` lists only the two served models (runbook 6 / the test-run checklist).

*Notes & pitfalls:*

- **The TLS fingerprint prompt comes back.** `/logout` deleted `gatewayTrust` along with the session, so the next connect is a fresh trust-on-first-use. The developer must confirm it against the fingerprint IT published (runbook 1) — do not let them click through it, since that prompt is the control that detects TLS interception in front of the gateway FQDN.
- `forceRemoteSettingsRefresh` **is not the culprit** and does not need removing. Its startup gate short-circuits as *succeeded* while logged out (remote managed settings only apply once a gateway session exists), so it does not block recovery. Resist the temptation to strip it from the GPO during triage — that would silently drop the model allowlist fleet-wide.
- **Opening** `api.anthropic.com` / `platform.claude.com` **in Zscaler also "fixes" it** and is the wrong lever: it widens egress permanently to work around a one-line local config, against the gateway-only posture the deployment is built on.
- **This is not user error to be trained away** — `/logout` is a documented, discoverable command. Treat recurrence as expected and keep this runbook reachable by the service desk.

*Prevention:* tell developers to sign out with `claude auth logout`, never `/logout`; that path leaves onboarding intact, so `claude` still starts and `/login` reconnects. The same trap applies to a **fresh install** on a locked-down laptop — a first-ever run has `hasCompletedOnboarding` unset and takes the identical preflight path — so exercise a clean profile during the test run before broad rollout rather than discovering it per-user.

*Rollback / recovery:* the edit is a single boolean in a backed-up file; restore `.claude.json.bak` to undo. Nothing server-side changes.

---

## 12. Teardown

*Trigger / Frequency:* Decommissioning the deployment (test account cleanup or end of life). Rare and deliberate. Status: **[NEEDS TEST-RUN CONFIRMATION]**.

*Order is the reverse of deploy:* `04 and 03 → 02 → 01`. The portal (`04`) and observability (`03`) stacks both import from `02` and are independent of each other — delete them (in either order, or in parallel) before the gateway. Since 2026-07-22 **03 no longer owns an ECS service or Cloud Map namespace** (the collector became a sidecar in the 02 gateway task), so its delete is simpler — no lingering collector ENIs, no discovery-service-before-namespace ordering to wait on. There is intentionally **no teardown script**; delete stacks explicitly so each destructive step is a conscious act. Downstream stacks import upstream exports, so an out-of-order delete fails on the export lock.

*Preconditions & the protection layers you must clear first:*

- **RDS** `DeletionProtection: true` blocks deleting the DB stack — disable it first (a stack update setting it false, or the console), and expect a **final snapshot** (`DeletionPolicy: Snapshot`).
- **ALB** `deletion_protection.enabled: true` blocks deleting the gateway stack — disable it first.
- The **stack policies** set by the deploy scripts deny `Update:Replace` / `Update:Delete` on `LoadBalancer` / `Database` during *updates*; they do not block `delete-stack`, but the deletion-protection flags above do. Clear those flags before deleting.

*Steps (exact commands):*

```
# 4) Download portal (if deployed)
aws cloudformation delete-stack --region "$AWS_REGION" --stack-name "$PORTAL_STACK_NAME"
aws cloudformation wait stack-delete-complete --region "$AWS_REGION" --stack-name "$PORTAL_STACK_NAME"

# 3) Observability
aws cloudformation delete-stack --region "$AWS_REGION" --stack-name "$OBS_STACK_NAME"
aws cloudformation wait stack-delete-complete --region "$AWS_REGION" --stack-name "$OBS_STACK_NAME"

# 2) Gateway (disable ALB deletion protection first, then delete)
aws cloudformation delete-stack --region "$AWS_REGION" --stack-name "$GATEWAY_STACK_NAME"
aws cloudformation wait stack-delete-complete --region "$AWS_REGION" --stack-name "$GATEWAY_STACK_NAME"

# 1) Database (disable RDS deletion protection first; a final snapshot is taken)
aws cloudformation delete-stack --region "$AWS_REGION" --stack-name "$DB_STACK_NAME"
aws cloudformation wait stack-delete-complete --region "$AWS_REGION" --stack-name "$DB_STACK_NAME"
```

*What survives deletion (verified against the templates'* `DeletionPolicy`*):*

- **KMS CMK** (`KmsKey`, 01) — `Retain`. Everything at rest was encrypted with it; retained so retained data stays readable. Schedule key deletion manually only after all encrypted artifacts are gone.
- **ALB access-logs bucket** (`AlbLogsBucket`, 02) — `Retain`.
- **Activity-archive bucket** (`ActivityArchiveBucket`, 03) — `Retain` (CMK-encrypted).
- **AMP workspace** (`Workspace`, 03) — `Retain`.
- **Portal artifacts bucket** (`ArtifactsBucket`, 04) — `Retain` (CMK-encrypted; holds the published release binaries).
- **Every CloudWatch log group, in every stack** — `Retain` (operator decision 2026-07-18: logs outlive stacks; a cfn-guard gate `log_groups_survive_teardown` enforces it). This covers the ECS task groups (gateway — which now also holds the collector sidecar's `otel` streams —

grafana, portal), the activity window, the portal download-audit group, the RDS `postgresql`/pgaudit export group (`/aws/rds/instance/${NAME_PREFIX}-store/postgresql`, pre-created by 01 with the CMK), and the db-admin Lambda groups (pre-created by 02 with the CMK). Deleting a retained group afterwards is a deliberate manual act. Note the redeploy consequence: the groups carry fixed names, so a later **re-create collides** with the retained groups and the new stack fails — export what you need, then delete them first (the test-run runbook §0 has the command list). The adopted groups (RDS postgresql, Lambda) additionally collide on the *first* deploy of this change into an account where the services already auto-created them.
- **RDS instance** (`Database`, 01) — `DeletionPolicy: Snapshot` → a **final snapshot** is taken; the running instance is removed. The snapshot persists.
- Everything else (ALB, ECS services/cluster, secrets, VPC endpoints, Lambdas, Firehose) is **deleted** with its stack.

*Verification:* all three `stack-delete-complete` waits return; `aws cloudformation describe-stacks` reports the stacks gone; confirm the retained buckets, CMK, AMP workspace, and final DB snapshot still exist if you intend to keep them, or clean them up explicitly.

*Rollback / recovery:* Redeploy from scratch per the test-run-runbook. Data recovery relies on the retained final RDS snapshot (restore per runbook 8) and the retained buckets.

*Notes & pitfalls:* Deleting a stack that another stack still imports from fails — always `04 and 03 → 02 → 01`, waiting for each delete to complete before the next tier. Retained resources are **not** free; account for the retained buckets, CMK, AMP workspace, and snapshots after teardown.